

Code Explanation Line by Line (Easy Language)

```
import numpy as np
```

Explanation:

- Imports NumPy library.
 - Used for:
 - Arrays
 - Matrix calculations
 - Mathematical operations
-

Store 4 Vectors

```
vectors = np.array([
    [1, 1, -1, -1],
    [-1, -1, 1, 1],
    [1, -1, 1, -1],
    [-1, 1, -1, 1]
])
```

Explanation:

- Stores 4 bipolar vectors.
- Values are:
 - 1
 - -1

These are training patterns stored in memory.

Initialize Weight Matrix

```
weights = np.zeros((4, 4))
```

Explanation:

- Creates 4×4 weight matrix.
- Initially all values are zero.

Example:

```
[[0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]
```

Weight Calculation

```
for i in range(4):
```

Explanation:

- Loops through rows.
-

for j in range(4):

Explanation:

- Loops through columns.
-

if i == j:

Explanation:

- Checks diagonal elements.
-

weights[i, j] = 0

Explanation:

- Sets diagonal weights to zero.

Because:

- Neuron cannot connect to itself.

Condition:

$$w_{ii} = 0$$

weights[i, j] = np.sum(vectors[i] * vectors[j])

Explanation:

- Calculates weight between vectors.
- Multiplies corresponding vector elements.
- Then adds all values.

This forms symmetric weight matrix.

Activation Function

def activation(x):

Explanation:

- Defines activation function.
-

return np.where(x >= 0, 1, -1)

Explanation:

- If value ≥ 0 :
 - Output = 1
- Else:
 - Output = -1

This is bipolar sign activation function.

Hopfield Function

```
def hopfield(input_vector, weights):
```

Explanation:

- Defines Hopfield recall function.
-

```
output_vector = activation(np.dot(weights, input_vector))
```

Explanation:

- Multiplies:
 - Weight matrix
 - Input vector

Formula:

$$Y = WX$$

- Then activation function converts output to:
 - 1
 - -1
-

```
return output_vector
```

Explanation:

- Returns recalled pattern.
-

Test Input

```
input_vector = vectors[0]
```

Explanation:

- Selects first stored vector as input.
-

Recall Pattern

```
output_vector = hopfield(input_vector, weights)
```

Explanation:

- **Network recalls stored pattern.**
-

Display Input

```
print("Input vector:")  
print(input_vector)
```

Explanation:

- **Displays input vector.**
-

Display Output

```
print("Output vector:")  
print(output_vector)
```

Explanation:

- **Displays recalled output vector.**
-

Expected Output

Input vector:
[1 1 -1 -1]

Output vector:
[1 1 -1 -1]

(Output may slightly vary.)

Viva Questions and Answers

1. What is Hopfield Network?

Hopfield Network is a recurrent neural network used for associative memory.

2. Who developed Hopfield Network?

It was developed by John Hopfield in 1982.

3. What is associative memory?

Memory that recalls stored patterns from noisy input.

4. What is recurrent neural network?

A neural network having feedback connections.

5. What are bipolar values?

Values:

- +1
 - -1
-

6. Why self-connections are removed?

Neuron should not connect to itself.

7. What is symmetric weight matrix?

Weights satisfying:

$$w_{ij} = w_{ji}$$

8. What is activation function?

Function deciding neuron output.

9. Which activation function is used?

Sign activation function.

10. Why NumPy is used?

For matrix and array operations.

11. What does np.dot() do?

Performs matrix multiplication.

12. What does np.where() do?

Checks condition and returns values.

13. What is weight matrix?

Matrix storing connection strengths.

14. What is pattern recall?

Recovering stored pattern from input.

15. What are applications of Hopfield Network?

Pattern recognition, optimization, image restoration.

16. Why Hopfield Network is fully connected?

Every neuron connects to every other neuron.

17. What is noisy input?

Corrupted or incomplete pattern.

18. What is discrete Hopfield Network?

Hopfield network using binary or bipolar values.

19. Why iterative process is used?

To stabilize output pattern.

20. Difference between Hopfield and Perceptron?

- Hopfield → Memory network
 - Perceptron → Classification network
-

Conclusion

Thus, we successfully implemented a Hopfield Neural Network using Python to store and recall 4 vectors. The network recalled stored patterns using associative memory and symmetric weight matrix concepts.